CS473 -Deep Learning
Module 1-Part1

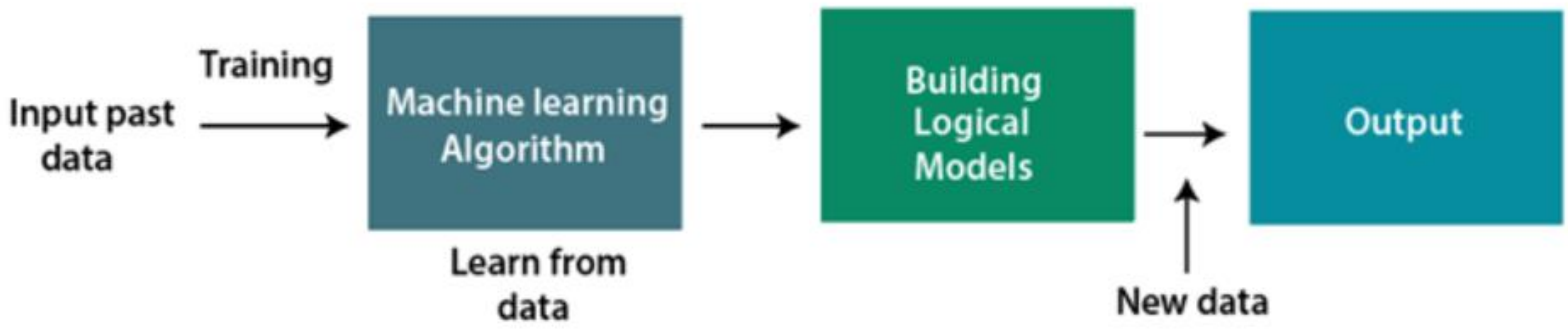
Module 1

- Machine Learning Basics: Learning Algorithms, Capacity, Over Fitting and Under fitting, Hyperparameters and Validation sets, Estimators, Bias and variance, Maximum Likelihood Estimation, BayesianStatistics, Supervised and Unsupervised Learning algorithms, SGD, Building a ML algorithm,

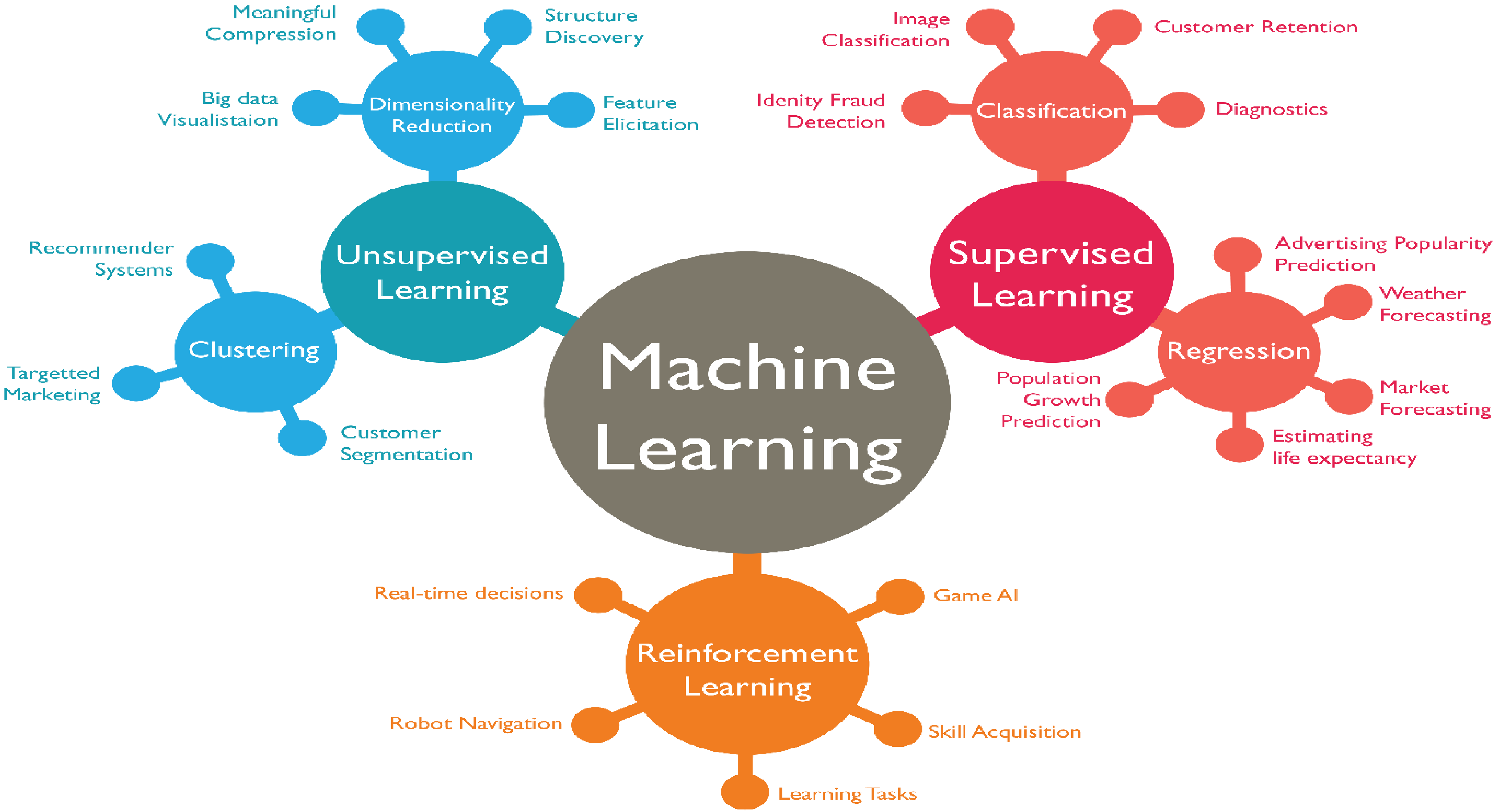
Machine Learning Basics

- A machine learning algorithm is an algorithm that is able to learn from data.
- what do we mean by learning?
- Mitchell (1997) provides the definition
- “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.”

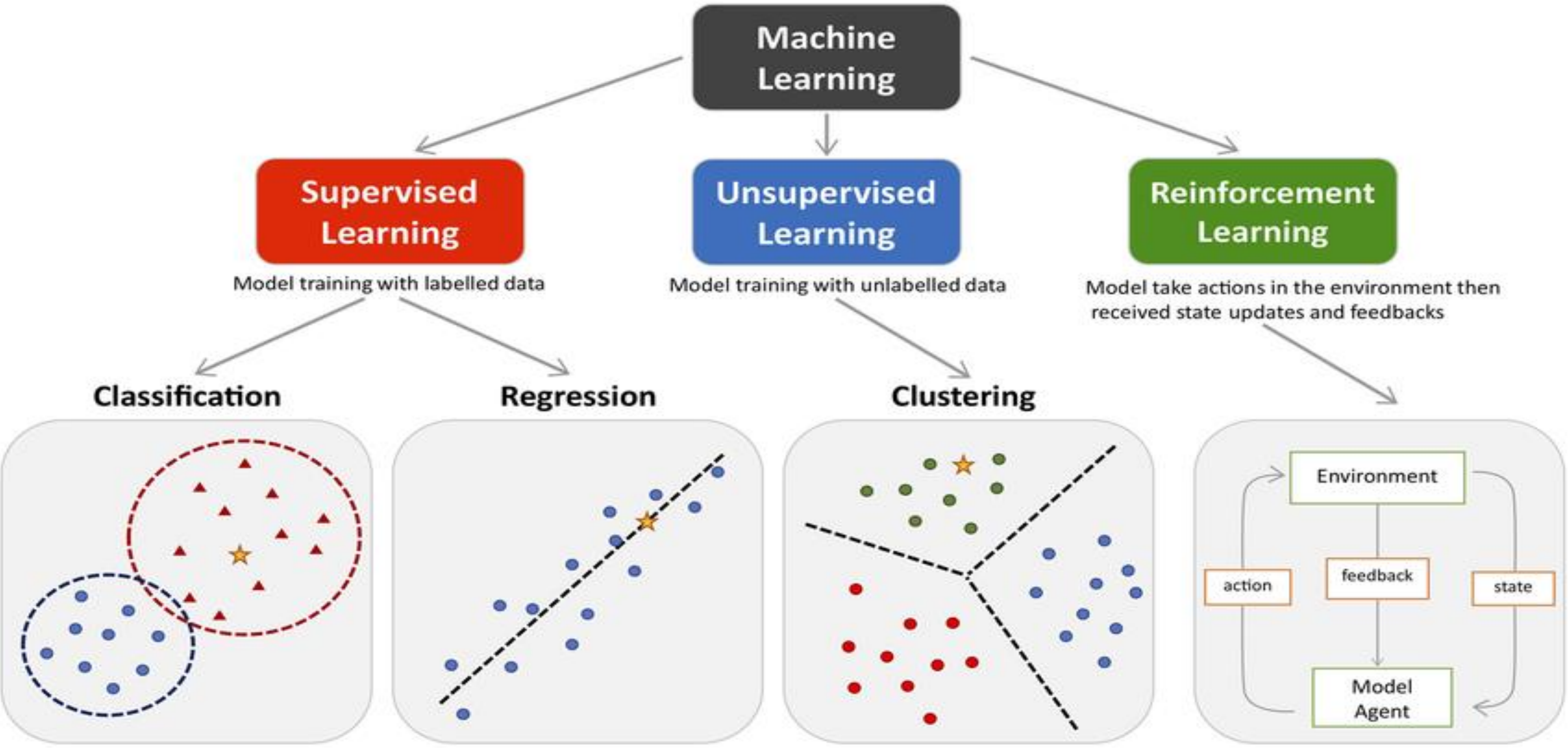
Machine Learning Basics



https://www.javatpoint.com/machine-learning



https://www.wordstream.com/blog/ws/2017/07/28/machine-learning-applications



https://www.researchgate.net/figure/The-main-types-of-machine-learning-Main-approaches-include-classification-and-regression_fig1_354960266

Machine Learning Basics

- Machine learning allows us to tackle tasks that are too difficult to solve with fixed programs written and designed by human beings.
- Learning is our means of attaining the ability to perform the task.
- For example, if we want a robot to be able to walk, then walking is the task.
- We could program the robot to learn to walk, or we could attempt to directly write a program that specifies how to walk manually.

Machine Learning Basics

- An example is a collection of features that have been quantitatively measured from some object or event that we want the machine learning system to process.
- We typically represent an example as a vector $x \in Rn$ where each entry xi of the vector is another feature.
- For example, the features of an image are usually the values of the pixels in the image.

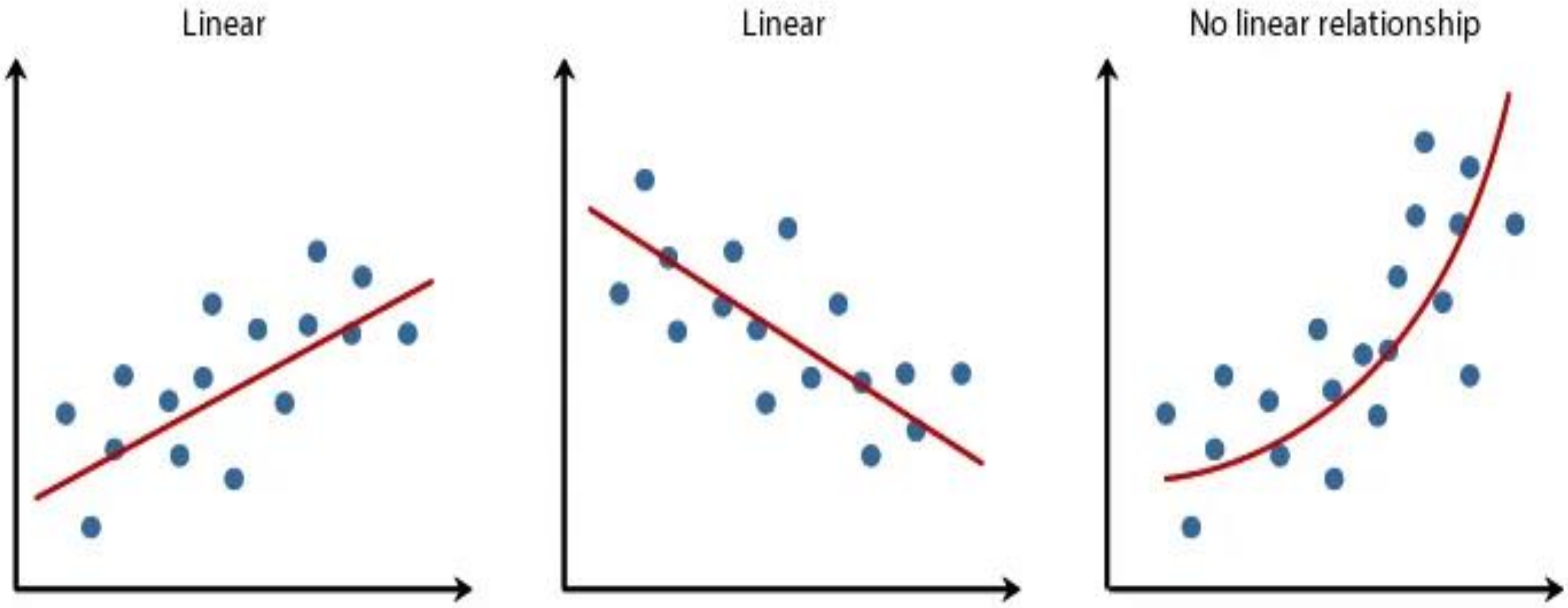
Machine Learning Basics

- Many kinds of tasks can be solved with machine learning.
- **Classification:**
- In this type of task, the computer program is asked to specify which of k categories some input belongs to.
- To solve this task, the learning algorithm is usually asked to produce a function $f : R^n \rightarrow \{1, . . . , k\}$.
- When $y = f(x)$, the model assigns an input described by vector x to a category identified by numeric code y.
- There are other variants of the classification task, for example, where f outputs a probability distribution over classes.

Machine Learning Basics

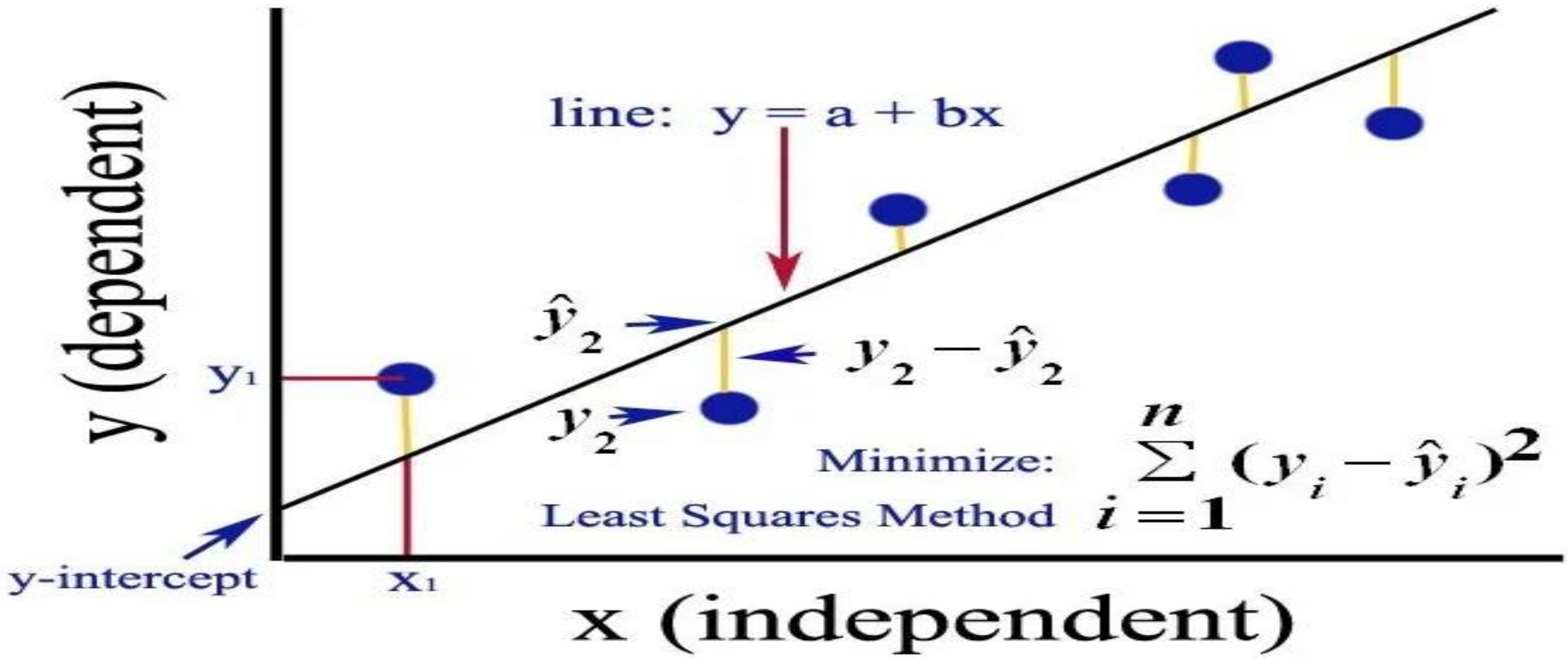
- **Classification:**
- An example of a classification task is object recognition, where the input is an image (usually described as a set of pixel brightness values), and the output is a numeric code identifying the object in the image.
- **Regression:** In this type of task, the computer program is asked to predict a numerical value given some input.
- To solve this task, the learning algorithm is asked to output a **function $f : R^n \rightarrow R$.** This type of task is similar to classification, except that the format of output is different.
- An example of a regression task is the prediction of the expected claim amount that an insured person will make (used to set insurance premiums),
- or the prediction of future prices of securities

Machine Learning Basics



https://medium.datadriveninvestor.com/linear-regression-and-its-mathematical-implementation-29d520a75ede

Machine Learning Basics



https://medium.datadriveninvestor.com/linear-regression-and-its-mathematical-implementation-29d520a75ede

Machine Learning Basics

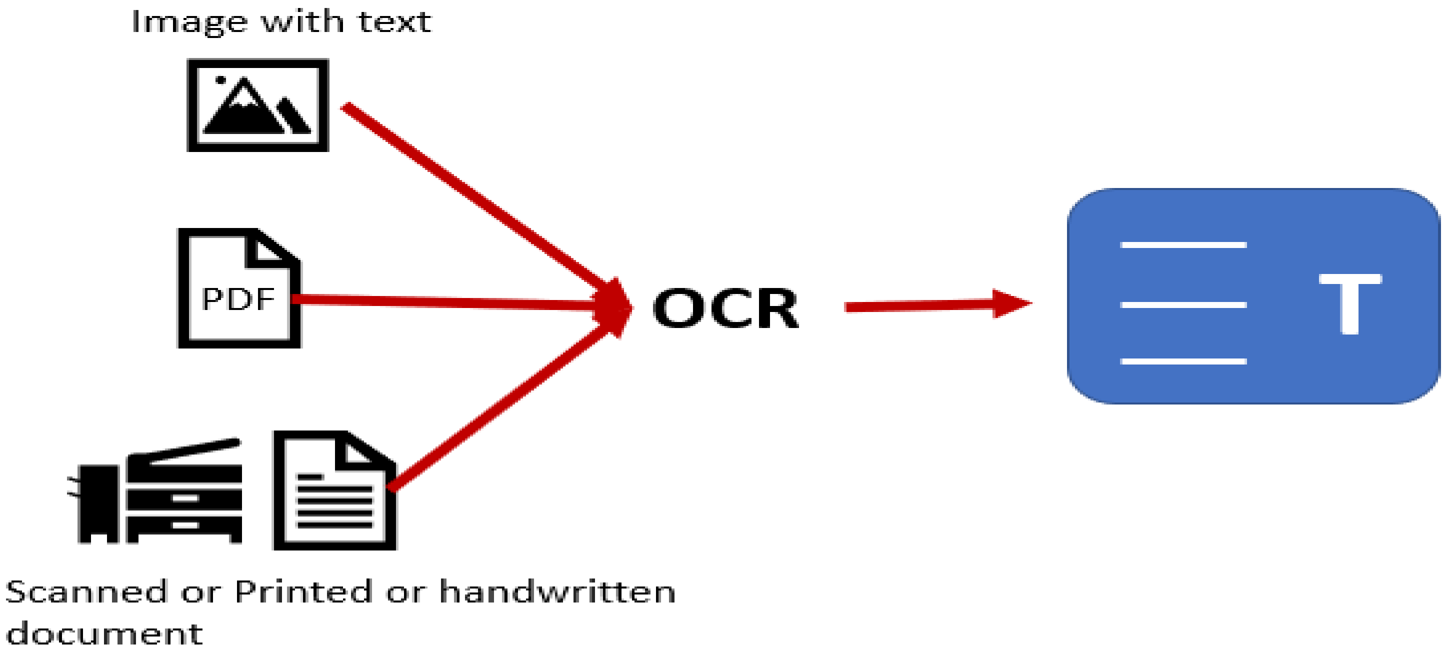
Transcription: In this type of task, the machine learning system is asked to observe a relatively unstructured representation of some kind of data and transcribe it into discrete, textual form.

For example, in **optical character recognition**, the computer program is shown a photograph containing an image of text and is asked to return this text in the form of a sequence of characters (e.g., in ASCII or Unicode format).

Another example is **speech recognition,** where the computer program is provided an audio waveform and emits a sequence of characters or word ID codes describing the words that were spoken in the audio recording.

Deep learning is a crucial component of modern speech recognition systems used at major companies including Microsoft, IBM and Google

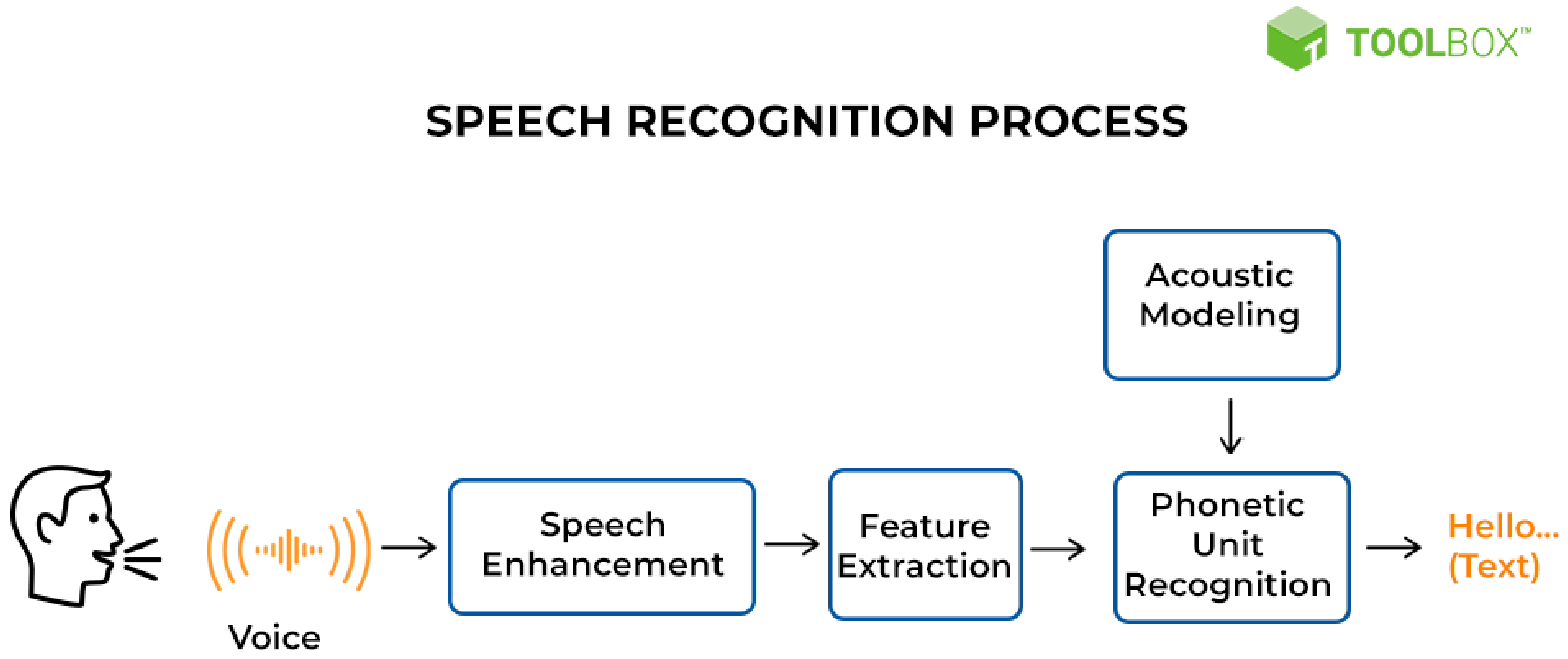
Machine Learning Basics



By transforming printed or handwritten text into data that computers can read, OCR is a cutting-edge technology that connects the physical and digital worlds

https://www.labellerr.com/blog/optical-character-recognition-overview/

Machine Learning Basics



https://www.spiceworks.com/tech/artificial-intelligence/articles/speech-recognition-software/

Machine Learning Basics

Machine translation:

This is commonly applied to natural languages, such as **translating from English to French**.

Structured output:

where the output is a vector with important relationships between the different elements.

One example is parsing—mapping a natural language sentence into a tree that describes its grammatical structure and tagging nodes of the trees as being verbs, nouns, or adverbs, and so on.

example, deep learning can be used to annotate the locations of roads in aerial photographs

For example, in image captioning, the computer program observes an image and outputs a natural language sentence describing the image .

These tasks are called structured output tasks because the program must output several values that are all tightly inter-related.

For example, the words produced by an image captioning program must form a valid sentence.

Machine Learning Basics

https://evergreen.team/articles/automatic-image-captioning.html

Machine Learning Basics

Anomaly detection:

An example of an anomaly detection task is credit card fraud detection.

By modeling your purchasing habits, a credit card company can detect misuse of your cards.

If a thief steals your credit card or credit card information, the thief’s purchases will often come from a different probability distribution over purchase types than your own

Machine Learning Basics

Synthesis and sampling:

In this type of task, the machine learning algorithm is asked to generate new examples that are similar to those in the training data.

Synthesis and sampling via machine learning can be useful for media applications where it can be expensive or boring for an artist to generate large volumes of content by hand.

For example, video games can automatically generate textures for large objects or landscapes, rather than requiring an artist to manually label each pixel.

For example, in a speech synthesis task, we provide a written sentence and ask the program to emit an audio waveform containing a spoken version of that sentence.

Machine Learning Basics

Imputation of missing values:

The algorithm must provide a prediction of the values of the missing entries.

Denoising:

The learner must predict the clean example x from its corrupted version x˜, or more generally predict the conditional probability distribution p(x | x˜).

Machine Learning Basics

Density estimation or probability mass function estimation:

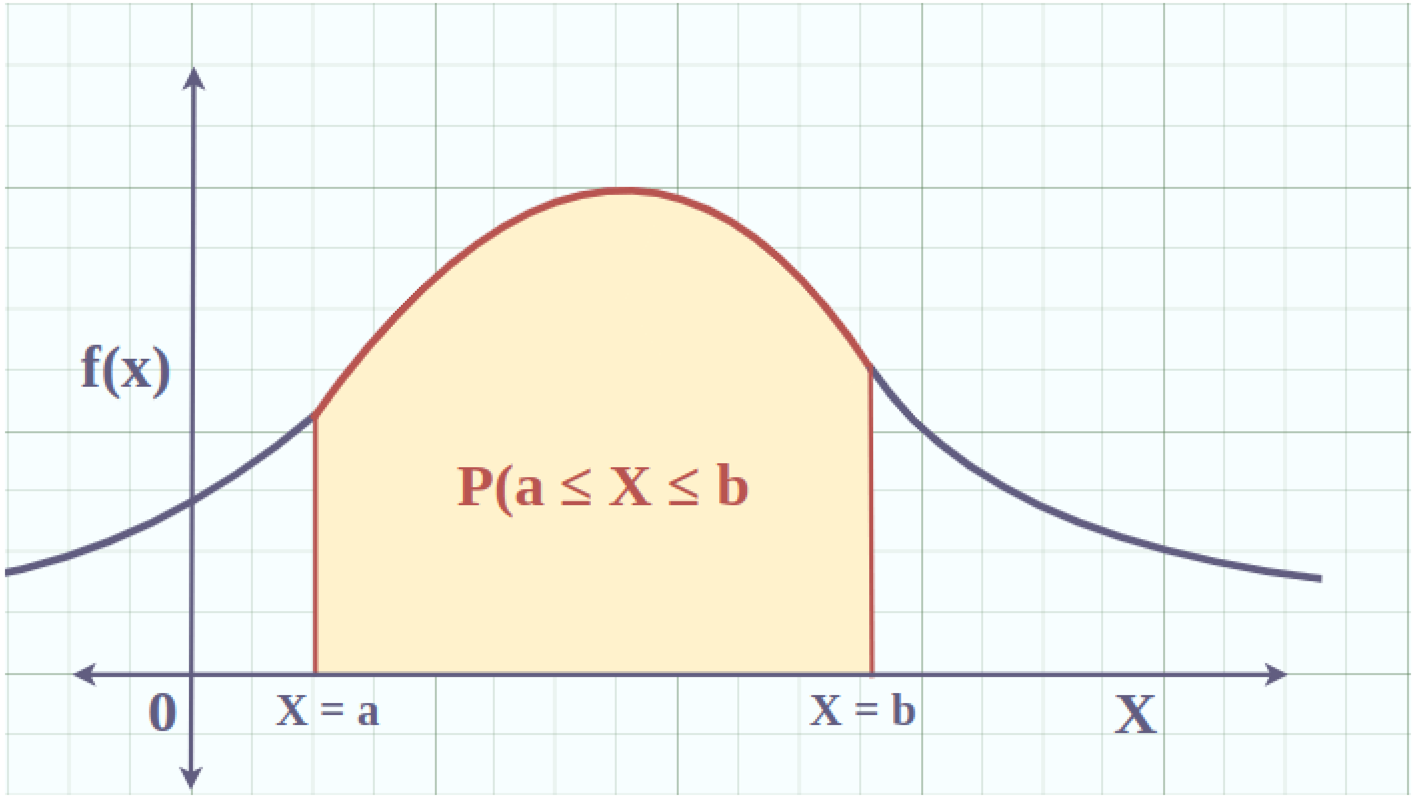
the machine learning algorithm is asked to learn a function pmodel : $R^n \rightarrow R$, where pmodel(x) can be interpreted as a probability density function (if x is continuous)

or a probability mass function (if x is discrete) on the space that the examples were drawn from.

the algorithm needs to learn the structure of the data it has seen.

It must know where examples cluster tightly and where they are unlikely to occur.

Machine Learning Basics



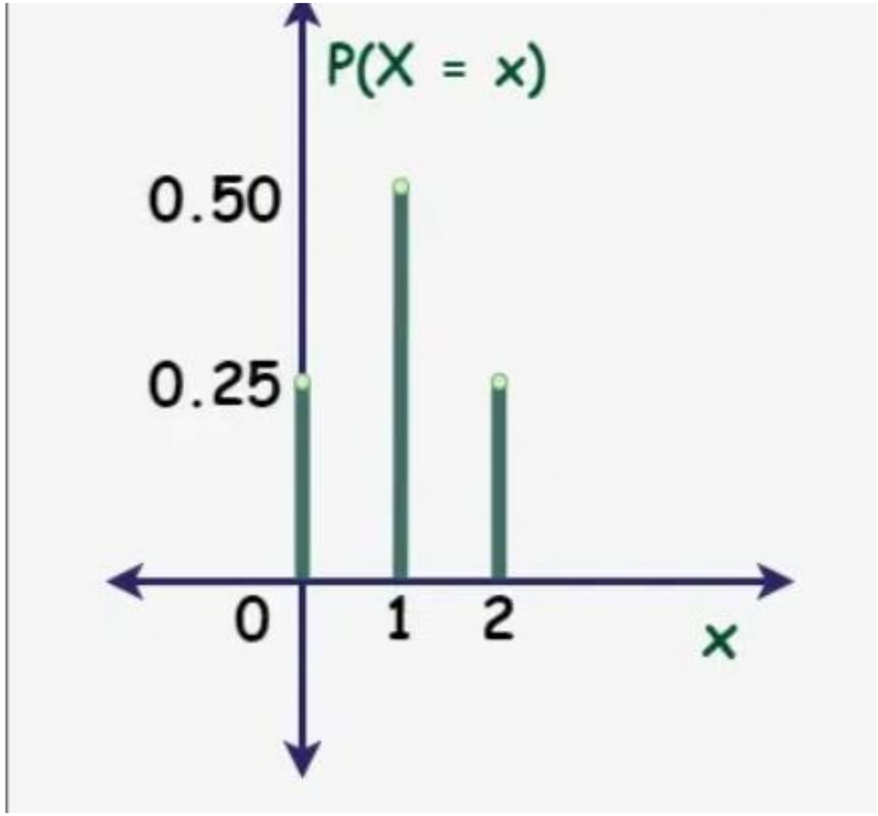
If X is continuous random variable and f(x) be the probability density function
The following graph gives the probability for X lying between interval a and b.

https://www.geeksforgeeks.org/probability-density-function/

Machine Learning Basics

x	0{HH}	1{HT, TH}	2{TT}
P(X = x)	1/4	2/4 = 1/2	1/4

Let a coin is tossed two times and X be the random variable representing the number of tails then, the probability mass function table for above event is given below



Probability-mass-function (PMF)

https://www.geeksforgeeks.org/probability-mass-function/

The Performance Measure

- In order to evaluate the abilities of a machine learning algorithm, we must design a quantitative measure of its performance.
- For tasks such as **classification**, classification with missing inputs, and transcription, we often measure the **accuracy of the model.**
- **Accuracy** is just the proportion of examples for which the model produces the correct output.
- We can also obtain equivalent **information by measuring the error rate**, the proportion of examples for which the model produces an incorrect output.
- We often refer to the error rate as the expected 0-1 loss.
- **The 0-1 loss on a particular**
- example is 0 if it is correctly classified
- and 1 if it is not.

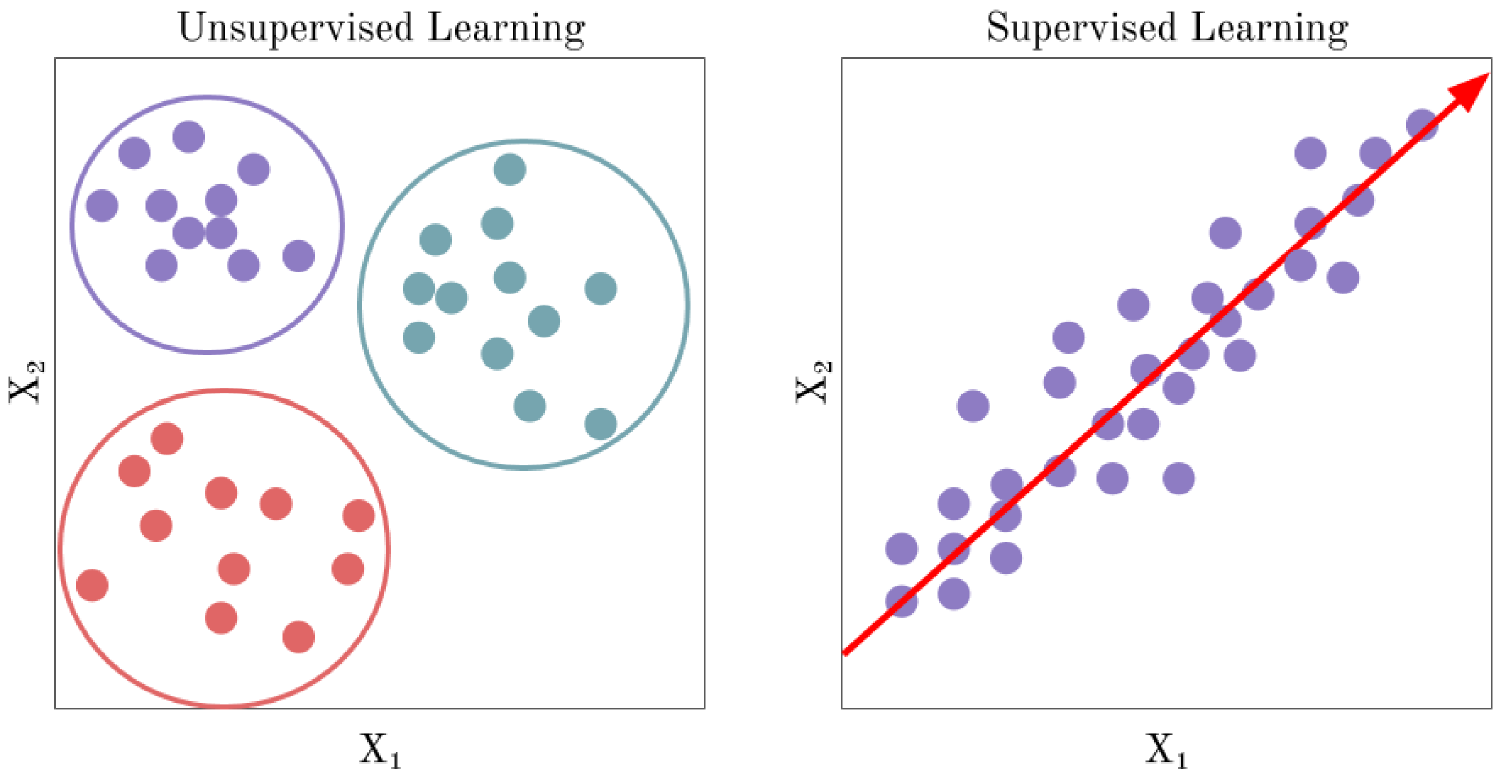
The Performance Measure, P

- For tasks such as **density estimation**, it does not make sense to measure accuracy, error rate, or any other kind of 0-1 loss.
- Instead, we must use a different performance metric that gives the model a continuous-valued score for each example.
- The most common approach is to report the **average log-probability** the model assigns to some examples.
- we are interested in how well the machine learning algorithm performs on data that it has not seen before,
- since this determines how well it will work when deployed in the real world.
- We therefore evaluate these performance measures using a **test set** of data that is separate from the data used for **training** the machine learning system.

The Experience, E

- Machine learning algorithms can be broadly categorized **as unsupervised or supervised** by what kind of experience they are allowed to have during the learning process.
- A dataset is a collection of many data points.
- **Unsupervised learning** algorithms experience a dataset containing many features, then learn useful properties of the structure of this dataset.
- In the context of deep learning, we usually want to learn the entire probability distribution that generated a dataset, whether explicitly as in density estimation or implicitly for tasks like synthesis or denoising.
- **clustering**, which consists of dividing the dataset into clusters of similar examples.

The Experience, E



https://towardsdatascience.com/a-brief-introduction-to-unsupervised-learning-20db46445283

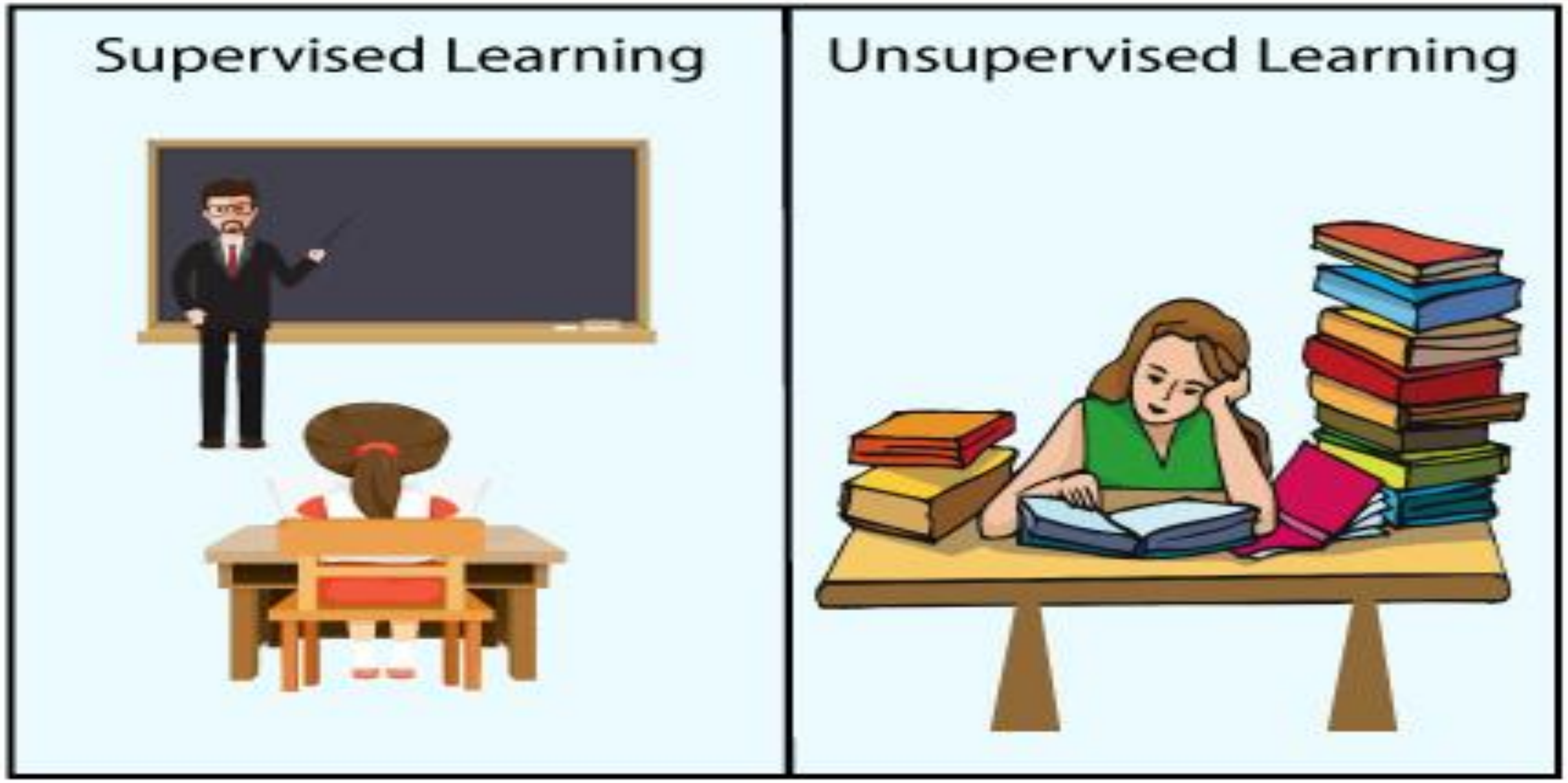
The Experience, E

- **Supervised learning** algorithms experience a dataset containing features, but each example is also associated with a **label or target**. For example, the Iris dataset is annotated with the species of each iris plant.
- **A supervised learning** algorithm can study the Iris dataset and learn to classify iris plants into three different species based on their measurements.
- **unsupervised learning** involves observing several examples of a random vector x, and attempting to implicitly or explicitly learn the **probability distribution p(x),** of that distribution.

The Experience, E

- **supervised learning** involves observing several examples of a random vector x and an associated value or vector y, and learning to predict y from x, usually by estimating **p(y | x).**
- The term supervised learning originates from the view of the target y being provided by an **instructor or teacher** who shows the machine learning system what to do.
- In **unsupervised learning**, **there is no instructor or teacher**, and the algorithm must learn to make sense of the data without this guide

The Experience, E



https://www.javatpoint.com/difference-between-supervised-and-unsupervised-learning

The Experience, E

- Many machine learning technologies can be used to perform both tasks.
- For example, the chain rule of probability states that for a vector x ∈ R^n, the joint distribution can be decomposed as
- P(x) = $\prod_{i=1}^{n} p(x_i \mid x_1, \ldots\ldots\ldots x_{i-1})$
- we can solve the ostensibly **unsupervised problem** of **modeling p(x)** by splitting it into **n supervised learning problems**.
- We can solve the **supervised learning problem** of learning p(y | x) by using traditional unsupervised learning technologies to learn the joint distribution p(x, y) and inferring
- p(y|x) = $\frac{p(x,y)}{\sum_{y'} p(x,y')}$

The Experience, E

- Traditionally, people refer to **regression, classification and structured output problems** as supervised learning.
- **Density estimation in support** of other tasks is usually considered unsupervised learning.
- **semisupervised learning,** some examples include a supervision target but others do not.
- In multi-instance learning, an entire collection of examples is labeled as containing or not containing an example of a class, but the individual members of the collection are not labeled

The Experience, E

- Some machine learning algorithms do not just experience a fixed dataset.
- For example, **reinforcement learning algorithms** interact with an environment, so there is a feedback loop between the learning system and its experiences.
- a dataset is a collection of examples, which are in turn collections of features.
- One common way of describing a dataset is with a design matrix.
- For instance, the Iris dataset contains 150 examples with four features for each example.
- This means we can represent the dataset with a design matrix $X \in \mathbf{R}^{150X4}$
- , where Xi,1 is the sepal length of plant i,
- Xi,2 is the sepal width of plant i, etc.

Linear Regression

- linear regression solves a regression problem.
- the goal is to build a system that can take a vector $x \in R^n$ as input and predict the value of a scalar $y \in R$ as its output.
- In the case of linear regression, the output is a linear function of the input.
- Let y^ be the value that our model predicts y should take on.
- We define the output to be $\hat{\boldsymbol{y}}$ = $\mathbf{w^t}$ $\mathbf{x}$
- where $w \in R^n$ is a vector of parameters

Linear Regression

- We define the output to be $\hat{y}$ $\mathbf{= w^t\ x}$
- Parameters are values that control the behavior of the system.
- wi is the coefficient that we multiply by feature xi before summing up the contributions from all the features.
- We can think of w as a set of weights that determine how each feature affects the prediction.
- If a feature xi receives a positive weight wi, then increasing the value of that feature increases the value of our prediction yˆ.
- If a feature receives a negative weight, then increasing the value of that feature decreases the value of our prediction.
- If a feature's weight is large in magnitude, then it has a large effect on the prediction.
- If a feature's weight is zero, it has no effect on the prediction.

Linear Regression

- We thus have a definition of our task T :
- to predict y from x by outputting $\hat{y}$ **= $\mathbf{w}^{t}$ x**
- Next we need a definition of our performance measure, P.
- Suppose that we have a design matrix of m example inputs that we will not use for training, only for evaluating how well the model performs.
- We also have a vector of regression targets providing the correct value of y for each of these examples.
- Because this dataset will only be used for evaluation, we call it the **test set**. We refer to the design matrix of inputs as $\mathbf{X}^{(test)}$
- and the vector of regression targets as $\mathbf{y}^{(test)}$

Linear Regression

- One way of measuring the performance of the model is to compute the mean squared error of the model on the test set.
- If yˆ(test) gives the predictions of the model on the test set, then the mean squared error is given by
- **MSE$_{\text{test}}$** $= \boldsymbol{\frac{1}{m}\sum_i\left(\hat{y}^{(test)} - y^{(test)}\right)_i^2}$
- Intuitively, one can see that this error measure decreases to 0
- when $\boldsymbol{\hat{y}^{(test)}}$ = $\boldsymbol{y^{(test}}$
- We can also see that
- MSE$_{\text{test}}$ $= \frac{1}{m}\sum_i\left\|\hat{y}^{(test)} - y^{(test}\right\|_2^2$
- so the error increases whenever the Euclidean distance between the predictions and the targets increases.

Linear Regression

- we need to design an algorithm that will **improve the weights w** in a way that **reduces MSEtest** when the algorithm is allowed to gain experience by observing **a training set** (X(train) , y(train)).
- Solution is just to minimize the **mean squared error** on the training set, **MSEtrain.**
- To **minimize MSEtrain**, we can simply solve for where **its gradient is 0:**
- $\nabla_{\boldsymbol{w}}\ \boldsymbol{MSE_{train}} = \mathbf{0}$
- $\nabla_{\boldsymbol{w}} \frac{\mathbf{1}}{\boldsymbol{m}} \sum_{\boldsymbol{i}} \left\| \widehat{\boldsymbol{y}}^{(\boldsymbol{train})} - \boldsymbol{y}^{(\boldsymbol{train})} \right\|_{\mathbf{2}}^{\mathbf{2}}$ **=0**
- $\frac{\mathbf{1}}{\boldsymbol{m}} \nabla_{\boldsymbol{w}} \sum_{\boldsymbol{i}} \left\| \boldsymbol{X}^{(\boldsymbol{train})} \boldsymbol{w} - \boldsymbol{y}^{(\boldsymbol{train})} \right\|_{\mathbf{2}}^{\mathbf{2}} = \mathbf{0}$

Linear Regression

$$\nabla_w \left(X^{(train)}w - y^{(train)}\right)^T \left(X^{(train)}w - y^{(train)}\right) = 0$$

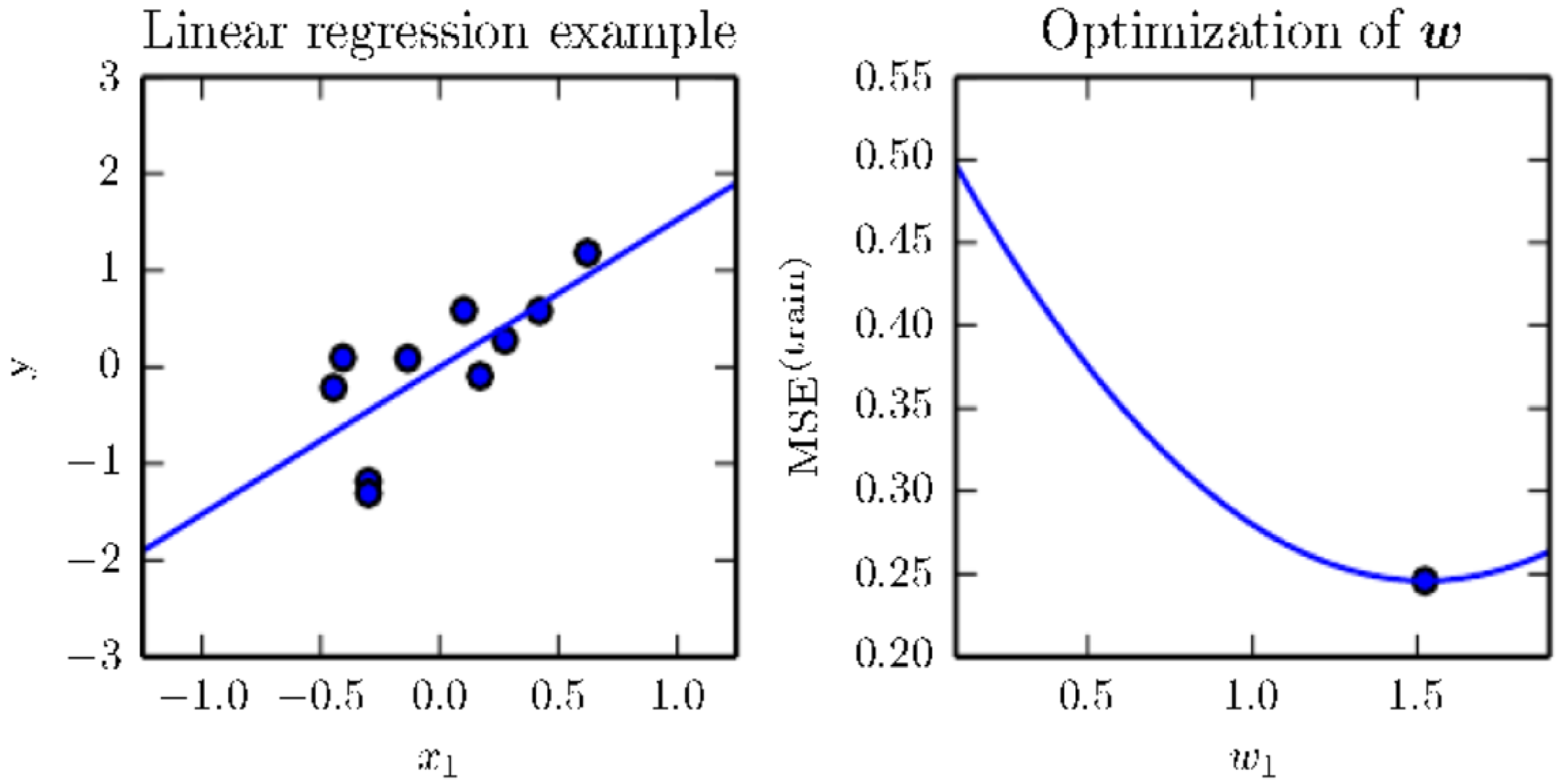
$$\nabla_w \left(w^T X^{(train)T} X^{(train)} w - 2w^T X^{(train)T} y^{(train)}\right) + y^{(train)T} y^{(train)} = 0$$

∇_w **= Partial derivative w.r.t w**

So ,

- $2X^{(train)T} X^{(train)} w - 2X^{(train)T} y^{(train)} = 0$
- **W=** $\left(X^{(train)T} X^{(train)}\right)^{-1} X^{(train)T} y^{(train)}$

normal equations



A linear regression problem, with a training set consisting of ten data points, each containing one feature. Because there is only one feature, the weight vector *w* contains only a single parameter to learn, *w*1 .
***(Left)*Observe that linear regression learns to set *w*1 such that the line *y* = *w*1*x* comes as close as possible to passing through all the training points.**
***(Right)*The plotted point indicates the value of *w*1 found by the normal equations, which we can see minimizes the mean squared error on the training set.**

Linear Regression

- It is worth noting that the term linear regression is often used to refer to
- a slightly more sophisticated model with one additional parameter—an **intercept term *b*.**
- In this model,
- $\hat{y} = \boldsymbol{w}^{\boldsymbol{T}}$ X + b
- so the mapping from parameters to predictions is still a linear function but the
- mapping from features to predictions is now an affine function.
- This extension to affine functions means that the plot of the model's predictions still looks like a line, but it need not pass through the origin